

Hooks — Automação determinística

Interceptadores de eventos do Claude Code — scripts que executam automaticamente em momentos-chave do ciclo de vida. Bloco 1: Stack de Extensão.

PreToolUse

PostToolUse

Notification

Stop

HTTP

O que são Hooks?

Hooks são interceptadores de eventos do Claude Code — scripts shell que executam automaticamente em pontos específicos: antes de usar uma ferramenta, depois, ao notificar ou ao encerrar.

Skills (Aula 1)

Instruções que guiam o comportamento do Claude. Ideal para: guias, templates e boas práticas. Probabilístico — o modelo decide seguir.

Hooks (Aula 2)

Scripts que interceptam ações do Claude. Ideal para: validações, logs e segurança. Determinístico — sempre executam.



A diferença essencial: uma Skill sugere; um Hook garante. Quando você precisa que algo sempre aconteça (ou nunca aconteça), use um Hook.

Os 4 tipos de Hooks

PreToolUse

Antes da ferramenta — pode bloquear.
Inspeciona os parâmetros e decide se permite ou bloqueia.

- ✓ Validar parâmetros
- ✓ Bloquear comandos perigosos
- ✓ Confirmar ações críticas

PostToolUse

Depois da ferramenta — pós-processa. Recebe o resultado e pode logar, validar ou transformar.

- ✓ Registrar logs de auditoria
- ✓ Validar o resultado gerado
- ✓ Formatar a saída

Notification

Quando o Claude notifica. Pode redirecionar para sistemas externos.

- ✓ Enviar alertas para Slack/Teams
- ✓ Registrar progresso em logs
- ✓ Notificar via webhook

Stop

Quando a sessão encerra.

- ✓ Gerar relatório de sessão
- ✓ Limpar arquivos temporários

Configurando no settings.json

📁 ~/.claude/settings.json (global) · .claude/settings.json (por projeto)

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "bash validate.sh"
      }]
    }],
    "PostToolUse": [{
      "matcher": "Write",
      "hooks": [{
        "type": "command",
        "command": "bash audit.sh"
      }]
    }]
  }
}
```

🔑 Tipo de Hook

PreToolUse, PostToolUse, Notification ou Stop

🎯 matcher

Nome da ferramenta que ativa o hook — ex.: Bash, Write, *

⚙️ type

command é o tipo padrão para executar scripts shell

🛠️ command

Comando shell a executar. Recebe o JSON do evento via stdin.



matcher: "*" ativa o hook para todas as ferramentas.

Interceptar e bloquear

O script recebe os parâmetros via stdin como JSON. Para bloquear, retorne exit code 2; para permitir, exit code 0.

```
#!/bin/bash - validate-bash.sh
INPUT=$(cat)
# extrai o comando que será executado
CMD=$(echo "$INPUT" | jq -r '.tool_input.command')

# bloqueia comandos perigosos
if echo "$CMD" | grep -qE "^(rm -rf|sudo rm)"; then
    echo "BLOQUEADO: comando perigoso" >&2
    exit 2 # bloqueia
fi

# auditoria + permite
echo "$(date): $CMD" >> ~/.claude/audit.log
exit 0 # permite
```

1 📁 Claude chama Bash

Decide executar um comando shell

2 🚫 Hook intercepta

PreToolUse dispara ANTES, recebe o JSON

3 🔍 Script analisa

Lê o JSON via stdin e checka regras

✓ Permitido / ✗ Bloqueado

exit 0 → Claude executa normalmente. exit 2 → Claude recebe erro e tenta outra abordagem.

Auditoria e pós-processamento

Executado após a ferramenta. Recebe entrada e resultado via stdin. Não bloqueia — mas registra, valida e reage.

```
#!/bin/bash - audit-writes.sh
INPUT=$(cat)
FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path')
# log estruturado (JSONL)
jq -n --arg file "$FILE" --arg user "$USER" \
  --arg time "$(date -u +%FT%TZ)" \
  '{action:"write",file:$file,time:$time}' \
  >> ~/.claude/audit.jsonl

# auto-formata se for .ts
if [[ "$FILE" == *.ts ]]; then
  npx prettier --write "$FILE"
fi
```

Log de auditoria

Registra escritas/leitura em JSONL para compliance

Auto-formatação

Roda Prettier/ESLint após o Claude escrever código

Webhook externo

POST para seu servidor quando arquivos críticos mudam

Roda testes

Executa testes relacionados assim que o arquivo é editado

Comunicar e encerrar

Notification → Slack

Ativado quando o Claude notifica o usuário. Redirecione para sistemas de comunicação.

```
INPUT=$(cat)
MSG=$(echo "$INPUT" | jq -r '.message')
curl -s -X POST "$SLACK_WEBHOOK" \
  -d "{\"text\":\"🤖 $MSG\"}"
```

Stop → relatório

Executado ao encerrar a sessão. Recebe o que foi feito. Ideal para relatórios e cleanup.

```
INPUT=$(cat)
TOOLS=$(echo "$INPUT" | jq '.tools_used')
echo "## Sessão · $TOOLS ferramentas" >> REPORT.md
rm -rf /tmp/claude-* # cleanup
```

✦ Notification: alertas de tarefas longas · Stop: relatório de alterações, limpeza e salvar contexto.

HTTP Hooks com servidores

Além de scripts shell, use type: "http" para enviar eventos direto a um servidor — ideal para integrações e microsserviços.

```
"PreToolUse": [{
  "matcher": "*",
  "hooks": [{
    "type": "http",
    "url": "http://localhost:8080/hook",
    "method": "POST",
    "headers": {
      "Authorization": "Bearer $TOKEN"
    },
    "timeout": 5000
  }]
}]
```

1 Claude usa ferramenta

Ex.: executa um comando Bash

2 Hook envia POST

JSON com detalhes para /hook

3 Servidor processa

Valida, registra ou rejeita

✓ 200 OK / ✗ 400-403

200 permite (retorna {} ou decision); 400/403 bloqueia a execução do Claude.



timeout padrão: 10s — o servidor deve responder rápido!

O que chega no Hook

Todo hook recebe um JSON via stdin. A estrutura varia por tipo.

```
// PreToolUse
{
  "session_id": "abc123",
  "tool_name": "Bash",
  "tool_input": {
    "command": "ls -la /src"
  },
  "hook_event": "PreToolUse"
}
```

```
// PostToolUse
{
  "session_id": "abc123",
  "tool_name": "Bash",
  "tool_input": {...},
  "tool_response":
    "file1.ts\nfile2.ts",
  "hook_event": "PostToolUse"
}
```

```
// Stop
{
  "session_id": "abc123",
  "total_cost": 0.0024,
  "tools_used": 12,
  "files_modified": [
    "src/app.ts", "README.md"
  ]
}
```



Campos-chave: session_id · tool_name · tool_input · tool_response · hook_event

Casos de uso reais

🛡️ PreToolUse — proteção de arquivos de produção

O Claude pode acidentalmente modificar `.env`, `.prod.*` ou configs críticas. O hook verifica o path antes de qualquer Write e bloqueia (exit 2) se estiver na lista negra: `if echo "$FILE" | grep -qE "(\.env|\.prod\.|secrets)"; then exit 2; fi`

✨ PostToolUse — auto-formatação de código

O Claude escreve código funcional mas fora do estilo do projeto. O hook roda Prettier após cada Write em arquivos `.ts/.tsx` — sem precisar lembrar: `if [[ "$FILE" =~ \.(ts|tsx)$ ]]; then npx prettier -w "$FILE"; fi`

🔔 Notification — alertas de tarefas longas no Slack

Tarefas longas rodam em segundo plano e o dev não sabe quando terminam. O hook envia mensagem no Slack toda vez que o Claude notifica — você fica informado sem monitorar o terminal.

Teste rápido

Você quer impedir que o Claude jamais execute `rm -rf`. Qual hook e qual exit code?

A) PreToolUse · exit 2

B) PostToolUse · exit 2

C) PreToolUse · exit 0

D) Stop · exit 1

Resposta: A — PreToolUse com exit 2. Só o PreToolUse roda antes da ferramenta e pode bloquear; o exit code 2 é o sinal que impede a execução. PostToolUse roda depois (tarde demais) e não bloqueia. exit 0 permitiria a ação.

Guardrails automáticos

PreToolUse

Intercepta ANTES. Bloqueia (exit 2) ou permite (exit 0). Segurança e validação.

PostToolUse

Roda APÓS. Recebe entrada + saída. Logs, formatação e webhooks.

Notification

Quando o Claude notifica. Redirecione para Slack, email ou webhook.

Stop

Ao encerrar. Relatórios, cleanup e salvar estado.

PRÓXIMA AULA → SUB-AGENTS: CONTEXTO ISOLADO, TOOL RESTRICTIONS E .CLAUDE/AGENTS/